

CS301

DATA STRUCTURE AND ALGORITHMS

LECTURE 8: APPLICATIONS OF LINKED LIST

Pandav Patel
Assistant Professor

Computer Engineering Department
Dharmsinh Desai University
Nadiad, Gujarat, India

OBJECTIVE

- To understand applications of linked list
- To introduce concept of generalized list

OVERVIEW

- 1 OBJECTIVE
- 2 LINKED LIST AS STACK AND QUEUE
 - Linked list as stack
 - Linked list as queue
- 3 ADDITION OF POLYNOMIALS USING LINKED LIST
 - Representation of polynomials using linked list
 - Addition of two polynomials using linked list
 - Ordered linked representation of polynomial
- 4 REAL WORLD APPLICATIONS

IMPLEMENT STACK USING LINKED LIST

- Can you implement stack using singly linked list?
 - Which end will you use for PUSH and POP operations?
 - What is the problem with using TAIL for POP?
- Can you implement stack using doubly linked list?
 - Which end will you use for PUSH and POP operations?
 - Is one end preferable over other?
- Is there any advantage in using circular (singly/doubly) linked list over non-circular (singly/doubly) linked list?

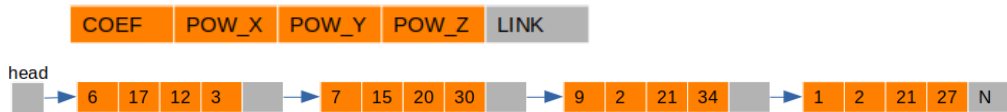
IMPLEMENT QUEUE USING LINKED LIST

- Can you implement queue using singly linked list?
 - Which end will you use for INSERT and DELETE operations?
 - What is the problem with using TAIL for DELETE?
- Can you implement queue using doubly linked list?
 - Which end will you use for INSERT and DELETE operations?
 - Is one end preferable over other?
- Is there any advantage in using circular (singly/doubly) linked list over non-circular (singly/doubly) linked list?

REPRESENTATION OF POLYNOMIALS USING LINKED LIST

■ $P(x, y, z) = 6x^{17}y^{12}z^3 + 7x^{15}y^{20}z^{30} + 9x^2y^{21}z^{34} + x^2y^{21}z^{27}$

- A single term of polynomial can be represented as one node of LL



■ $Q(x, y, z) = 9x^{15}y^{20}z^{30} - 6x^{17}y^{12}z^3 + 16x^3y^4z^2 + 17x^3$



- How to find $R(x, y, z) = P(x, y, z) + Q(x, y, z)$?

- What would be the time complexity of your approach?
■ Can we reduce time complexity by storing the terms in order?

REPRESENTATION OF POLYNOMIALS USING LINKED LIST (CONT...)

- Terms of polynomial can be stored in descending order
- Term $D_1X^{A_1}Y^{B_1}Z^{C_1}$ will precede term $D_2X^{A_2}Y^{B_2}Z^{C_2}$ if
 - $A_1 > A_2$ or
 - $A_1 = A_2$ and $B_1 > B_2$ or
 - $A_1 = A_2$ and $B_1 = B_2$ and $C_1 > C_2$
 - Assumption: For a given polynomial following situation will never arise
 - $A_1 = A_2$ and $B_1 = B_2$ and $C_1 = C_2$
- In previous slide, polynomial P is already in descending order, but Q is not

ADDITION OF TWO POLYNOMIALS USING LINKED LIST

CASES

- Polynomial with zero terms
 - Both polynomials have zero terms
 - One of the two polynomials has zero terms
- Number of terms are same
- Number of terms are different
- Regarding terms
 - A term in P and a term in Q may have same power
 - For a term in P there may not be term with same power in Q
 - For a term in Q there may not be term with same power in P
 - Addition of two terms with same power from P and Q may result in zero coefficient

ADDITION OF TWO POLYNOMIALS USING LINKED LIST (CONT...)

STEPS

- 1 Set CURRP and CURRQ to point to first terms of P and Q resp.
- 2 Repeat thru step 4 while there are terms left for processing in both polynomials
- 3 Obtain values for terms pointed by CURRP and CURRQ
- 4 If powers of both terms are equal (same power of all vars) then
 - If terms do not cancel (resultant coeff not zero)
 - Insert sum of terms at end of result
 - Advance CURRP and CURRQ to point to next terms of respective polynomials

Else if power of term pointed by CURRP $>$ power of term pointed by CURRQ

 - Insert term pointed by CURRP at end of result
 - Advance CURRP to point to next term of P

Else

 - Insert term pointed by CURRQ at end of result
 - Advance CURRQ to point to next term of Q
- 5 Append remaining terms from non-empty polynomial to result and return

Algorithm: POLLY_ADD(P, Q)

Add polynomials represented by P and Q.

P, Q: Pointers to first nodes of linked list representing polynomials to be added.

RH, RT: Pointer to first node and last node of result.

Assumptions: RH, RT are passed by ref to INS_AT_TAIL

CURRP and CURRQ: Pointers to nodes representing current term.

A1, A2, B1, B2, C1, C2, D1, D2: Temporary variables.

1. [Initialize]
CURRP $\leftarrow$ P
CURRQ $\leftarrow$ Q
RH $\leftarrow$ RT $\leftarrow$ NULL
2. [End of any polynomial?]
Repeat thru step 4 while CURRP $\neq$ NULL and CURRQ $\neq$ NULL
3. [Get values for current terms of P and Q]
A1 $\leftarrow$ POW_X(CURRP)
A2 $\leftarrow$ POW_X(CURRQ)
B1 $\leftarrow$ POW_Y(CURRP)
B2 $\leftarrow$ POW_Y(CURRQ)
C1 $\leftarrow$ POW_Z(CURRP)
C2 $\leftarrow$ POW_Z(CURRQ)
D1 $\leftarrow$ COEF(CURRP)
D2 $\leftarrow$ COEF(CURRQ)

4. [Compare terms and add to result]
If A1 = A2 and B1 = B2 and C1 = C2 then
If D1 + D2 $\neq$ 0 then
INS_AT_TAIL(A1, B1, C1, D1 + D2, RH, RT)
CURRP $\leftarrow$ LINK(CURRP)
CURRQ $\leftarrow$ LINK(CURRQ)
Else if (A1 > A2) or ((A1 = A2) and (B1 > B2))
or ((A1 = A2) and (B1 = B2) and (C1 > C2)) then
INS_AT_TAIL(A1, B1, C1, D1, RH, RT)
CURRP $\leftarrow$ LINK(CURRP)
Else
INS_AT_TAIL(A2, B2, C2, D2, RH, RT)
CURRQ $\leftarrow$ LINK(CURRQ)
5. [Terms remaining in any of the polynomials?]
If CURRP $\neq$ NULL then
LINK(RT) $\leftarrow$ COPY(CURRP)
Else if CURRQ $\neq$ NULL then
LINK(RT) $\leftarrow$ COPY(CURRQ)
return(RT)

ADDITION OF TWO POLYNOMIALS USING LINKED LIST (CONT...)

DISCUSSION

- What if in step 4, $D1 + D2 \neq 0$ is false?
- Why is step 5 required?
- Check if all cases discussed in earlier slide are working fine with above algorithm
 - Will it work fine when both P and Q has zero terms (NULL)?
 - Yes!
 - Will it work fine if either P or Q has zero terms (NULL) and other polynomial is non-empty (has at-least one term)?
 - It will fail. Can you fix it?

INSERT POLYNOMIAL TERM IN ORDERED LINKED LIST

CASES

- Empty list
- Non-empty list
 - Insert node at **front** end of the linked list
 - Insert node at the **rear** end of the linked list
 - Insert node in the **middle** of the linked list

INSERT POLYNOMIAL TERM IN ORDERED LINKED LIST (CONT...)

STEPS

- 1 Create a new node
- 2 Initialize node fields (POW_X, POW_Y, POW_Z and COEF)
- 3 Handle empty list case
- 4 Handle insert at front end
- 5 Find predecessor of node to be inserted
- 6 Add new node behind its predecessor and return

Algorithm: PINSERT(NX, NY, NZ, NCOEF, HEAD)

Assumptions: Node with given value of NX, NY, NZ does not exist in the list.

Add new node with NX, NY, NZ and NCOEF in ordered (descending) list.

HEAD: Pointers to first node of linked list

NEW: Pointer to newly created node

CURR: Temporary pointers to node

A, B, C: Temporary variables.

1. [Create a new node]
 NEW $\leftarrow$ Create a new node
 If NEW = NULL then
 Write("New node not created")
 return(HEAD)
2. [Initialize node fields]
 POW_X(NEW) $\leftarrow$ NX
 POW_Y(NEW) $\leftarrow$ NY
 POW_Z(NEW) $\leftarrow$ NZ
 COEF(NEW) $\leftarrow$ NCOEF
3. [Handle empty list case]
 If HEAD = NULL then
 LINK(NEW) $\leftarrow$ NULL
 return(NEW)

4. [Does new node precede first node in the list?]
 A $\leftarrow$ POW_X(HEAD)
 B $\leftarrow$ POW_Y(HEAD)
 C $\leftarrow$ POW_Z(HEAD)
 If (NX > A) or ((NX = A) and (NY > B))
 or ((NX = A) and (NY = B) and (NZ > C)) then
 LINK(NEW) $\leftarrow$ HEAD
 return(NEW)
5. [Find predecessor of new node to be inserted]
 CURR $\leftarrow$ HEAD
 While LINK(CURR) $\neq$ NULL do
 A $\leftarrow$ POW_X(LINK(CURR))
 B $\leftarrow$ POW_Y(LINK(CURR))
 C $\leftarrow$ POW_Z(LINK(CURR))
 If (NX < A) or ((NX = A) and (NY < B))
 or ((NX = A) and (NY = B) and (NZ < C)) then
 CURR $\leftarrow$ LINK(CURR)
 Else
 Exitloop
6. [Add new node behind predecessor and return]
 LINK(new) $\leftarrow$ LINK(CURR)
 LINK(CURR) $\leftarrow$ NEW
 return(HEAD)

INSERT POLYNOMIAL TERM IN ORDERED LINKED LIST (CONT...)

DISCUSSION

- What if node with given powers already exist in the list?
 - Algorithm as it is will add new term with same powers
- Can you rewrite this algorithm so that it works fine when node with given powers already exist in the list? (i.e. for some term present in the list, $A = NX$ and $B = NY$ and $C = NZ$)
 - When Powers are same then coefficient (NCOEF) should be added to the coefficient of the existing term

REAL WORLD APPLICATIONS OF LINKED LIST

- To implement Undo and Redo functionalities
- To provide navigation of previous and next photo/link in photo viewer/web browser
- Task scheduling by operating system in round robin fashion
- Playlist of songs with next, previous, loop functionality
- Let us implement a playlist where user can set his/her liking of a song in range 1 to 1 million. Following functionalities are expected. Which data structure should be preferred? Think of complexity for each of these operations.
 - Add new song to the list
 - Start listening from most favourite song to least favourite song
 - Delete least favourite song
 - Add new song to the list (liking of it can range from 1 to 1 million)
 - Play songs in a loop

REAL WORLD APPLICATIONS OF LINKED LIST (CONT...)

- What if we want to add following functionality to our solution in previous slide?
 - Start listening from least favourite song to most favourite song
- What if we want to add one more functionality to our solution?
 - Play songs randomly. Make sure that song is not repeated. (Question asked to me in Google interview in 2012)

GENERALIZED LIST (OFF-TOPIC)

- A generalized list is a finite sequence of $n \geq 0$ elements, $a_0, a_1, a_2, \dots, a_{n-1}$, where a_i is either an atom or a list.
 - $A = (a_0, a_1, a_2, \dots, a_{n-1})$
- Examples
 - $(11, 12, 31, 41, 5, 16)$
 - $(1, 21, (13, 41), 51, 16)$
 - $(1, 21, (13, (12, 12, 13, 41), (12, 5), 7), 51, 16)$

